SMS Spam Detection System Using NLP

A Project Report

submitted in partial fulfillment of the requirements

of

AICTE Internship on AI: Transformative Learning
with
TechSaksham – A joint CSR initiative of Microsoft & SAP

by

Vaishnavi Kainthola, vkainthola206@gmail.com

Under the Guidance of

Rathod Jay

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to Rathod Jay for their invaluable guidance, encouragement, and unwavering support throughout the four weeks of this project. Their expertise and constructive feedback have been instrumental in shaping the project's direction and ensuring its successful completion. Their willingness to share knowledge and help at every stage has truly inspired me and enhanced my learning experience.

I am also deeply thankful to Edunet Foundation for providing me with the opportunity and resources to undertake this project under Techsaksham program. The facilities, tools, and conducive environment greatly contributed to my ability to work effectively and achieve the desired results.

Lastly, I am grateful to my family and friends for their encouragement and understanding during this period. Their moral support kept me motivated and focused on achieving the project's objectives.

Thank you all for being part of this endeavor!

ABSTRACT

Problem Statement

Unsolicited spam emails are a growing problem, overwhelming users and posing potential security risks. A reliable system to classify emails as spam or ham (not spam) is essential to enhance user productivity and safeguard information.

Summary

The SMS Spam Detection Model employs NLP and machine learning to classify emails as "spam" or "ham." Using **CountVectorizer** for text preprocessing and a **Multinomial Naive Bayes** classifier, it provides accurate predictions. The model features both a GUI built with **Tkinter** and a user-friendly web interface via **Streamlit**, where users can input emails for classification. Voice feedback enhances the desktop experience, while the Streamlit app ensures a seamless online user experience, making it versatile and interactive.

Objectives

1. Develop an accurate and efficient SMS/email spam detection model using machine learning.
2. Create an intuitive user interface for real-time email classification.
3. Incorporate voice feedback and web-based accessibility to improve user interaction.

Methodology

The model utilizes a labeled dataset of emails, where messages are categorized as spam or ham. Preprocessing involves cleaning text data and vectorizing it using **CountVectorizer**. The core classifier is a **Multinomial Naive Bayes** model, trained on an 80/20 train-test split for optimal performance. The model is then integrated with:

1. A **Tkinter-based GUI** offering voice feedback for desktop users.
2. A **Streamlit-based web app**, allowing users to classify emails online.

Key Results

The model achieves high accuracy in detecting spam emails. Both interfaces ensure easy input and clear classification results, with the web app providing broader accessibility.

Conclusion

This project demonstrates the effective application of machine learning for spam detection, combining accuracy with versatile interfaces to address user needs and improve email management.

TABLE OF CONTENT

LIST OF FIGURES

CHAPTER 1

Introduction

1.1Problem Statement:

The problem addressed is the detection of spam emails—unsolicited, irrelevant, or malicious messages that flood inboxes daily. Spam emails not only waste users' time but also pose significant risks, including phishing attacks, spreading malware, and compromising sensitive information. Traditional manual filtering is inefficient and error-prone, making automated spam detection a critical need.

This problem is significant because:

Volume and Frequency: With billions of emails exchanged daily, a large proportion are spam, causing digital clutter and reducing productivity.

Security Risks: Spam emails often serve as vectors for cyberattacks, including phishing, ransomware, and identity theft, endangering both individuals and organizations.

Economic Impact: Spam incurs substantial costs globally, including lost time, bandwidth consumption, and financial losses from fraud.

User Experience: A clutter-free inbox improves the efficiency and experience of email communication.

By addressing this issue with a robust, machine-learning-powered spam detection system, the project provides an automated, scalable, and accurate solution to a persistent and impactful challenge in digital communication

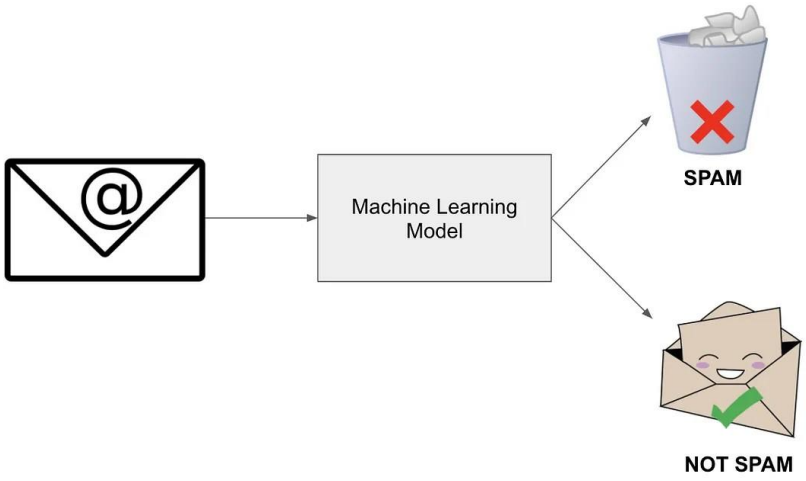

Fig 1 Spam Detection

1.2Motivation:

This project was chosen to tackle the pervasive problem of spam emails, which affect billions of users worldwide. With the increasing reliance on digital communication, an automated and efficient spam detection system has become essential. The project's aim is to use machine learning to provide a scalable, accurate, and user-friendly solution to manage email clutter and mitigate associated security risks.

Potential Applications

1. **Email Service Providers**: Enhance email filtering systems for platforms like Gmail, Outlook, and Yahoo Mail.
2. **Enterprise Solutions**: Help businesses safeguard against phishing and spam, improving security and productivity.
3. **Individual Users**: Provide tools for better email management, saving time and reducing frustration.
4. **Cybersecurity**: Detect malicious emails, protecting users from fraud, malware, and data breaches.
5. **Education and Awareness**: Serve as a learning tool for understanding machine learning applications in text classification.

Impact

1. **Improved Productivity**: Reduces time spent manually filtering spam emails.
2. **Enhanced Security**: Lowers exposure to cyber threats, protecting personal and organizational data.
3. **Broader Accessibility**: The integration of GUI and web apps ensures usability across different platforms and user groups.
4. **Economic Savings**: Minimizes losses associated with spam and cyberattacks.

This project demonstrates the transformative potential of machine learning in solving real-world problems.

1.3Objective:

- **Develop an Accurate Spam Detection Model**
  Create a machine learning model capable of accurately classifying emails as spam or ham (not spam) based on their content.

- **Implement Effective Text Preprocessing**
  Utilize techniques like text cleaning and vectorization to convert unstructured email text into structured data suitable for machine learning.

- **Utilize a Robust Classifier**
  Leverage the **Multinomial Naive Bayes** algorithm for its proven effectiveness in text classification tasks.

- **Ensure Real-Time Usability**
Build a user-friendly desktop application using **Tkinter** and a web-based application using **Streamlit** for seamless interaction.

- **Incorporate Voice Feedback**
Enhance the desktop application with voice feedback to improve user experience and accessibility.

- **Enhance Accessibility**
Provide a solution accessible to both individual users and organizations through intuitive interfaces and scalable deployment options.

- **Achieve Scalability and Reliability**
Design a system capable of handling varying email volumes while maintaining performance and accuracy.

- **Raise Awareness of Spam Risks**
Educate users about the risks posed by spam and the importance of automated email filtering.

1.4Scope :

1. **Spam Detection**: The model focuses on classifying emails as either spam or ham (not spam) using machine learning techniques.
2. **Text-Based Classification**: It uses the email content (text) as input for classification, ensuring simplicity and ease of implementation.
3. **Applications**:
    - Desktop application using **Tkinter** for local use with voice feedback.
    - Web-based application using **Streamlit** for online, real-time classification.
4. **Use Cases**: The solution can be utilized by individual users, businesses, and email service providers to enhance email management and security.
5. **Scalability**: The solution can be extended to handle large-scale email datasets with minor adjustments.
6. **User Interaction**: Provides an intuitive interface to make the system user-friendly for non-technical users.

Limitations

1. **Content-Based Classification Only**: The model relies solely on the email's text content and does not consider metadata like sender information or email headers.
2. **Language Dependency**: The model may not perform well with emails written in languages not present in the training dataset.
3. **Generalization**: Performance depends on the dataset used for training. It may need retraining for different datasets or evolving spam patterns.
4. **Limited Context Understanding**: It cannot identify advanced spam techniques, such as image-based spam or context-specific phishing attempts.
5. **Real-Time Processing**: While suitable for moderate volumes, handling extremely large-scale email datasets might require additional computational resources.
6. **Voice Feedback Restriction**: Voice feedback is available only in the desktop application, limiting accessibility to specific platforms.

CHAPTER 2

Literature Survey

2.1Relevant literature / Previous work

1. **Spam classification using rule-based systems**
   They work by sorting the text into distinct groups using handcrafted linguistic rules. The entering text is classified using semantic factors based on its content. Certain terms can help you evaluate whether or not a text message is spam. The spam text has a few distinctive phrases that help differentiate it from non-spam language. The document is classified as spam when the number of spam words in it exceeds the number of non-spam (ham) terms.

2. **Machine Learning (ML) techniques for spam classification**
   To detect spam reviews, a variety of machine learning techniques have been deployed. There are two types of machine learning: supervised learning and unsupervised learning, both of which are extensively utilized in NLP applications. Algorithms like Naive Bayes, Support Vector Machines (SVM), and Decision Trees have been widely used for email spam classification due to their ability to generalize well across unseen data. Jancy Sickory Daisy & Rijuvana Begum (2021) used the Nave Bayes method and the Markov Random Field to circumvent the limitations of other filtering algorithms.

3. **Hybrid approach for spam classification**
   To increase spam classification performance, hybrid spam detection systems combine a machine learning-based classifier with a rule-based approach. Their rule-based solution involves assigning suitable weights to spam material and generating a matrix that is then submitted to a classifier. Venkatraman, Surendiran & Arun Raj Kumar (2020) employed a semantic similarity technique combined with the Naive Bayes (NB) machine learning algorithm to classify spam material.

4. **Deep learning (dl) approaches for spam classification**
   Deep learning models are gaining popularity among NLP researchers due to their ability to solve challenging problems (Kłosowski, 2018; Torfi et al., 2020). Deep learning is based on the idea of building a very large neural network inspired by brain activities and training it using a massive amount of data. They can cope with the scalability issue and extract the features from the data automatically. The most popular deep learning models among NLP researchers are Convolutional Neural Networks (CNN) and Long Short Tern Memory (LSTM) networks.

2.2Machine Learning Models

2.2.1 Decision Tree:

Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems but mostly it is preferred for solving Classification problems. It is a true-structured classifier, where internal nodes represent the features of the dataset, branches represent the decision rules, and each leaf node represents the outcome.

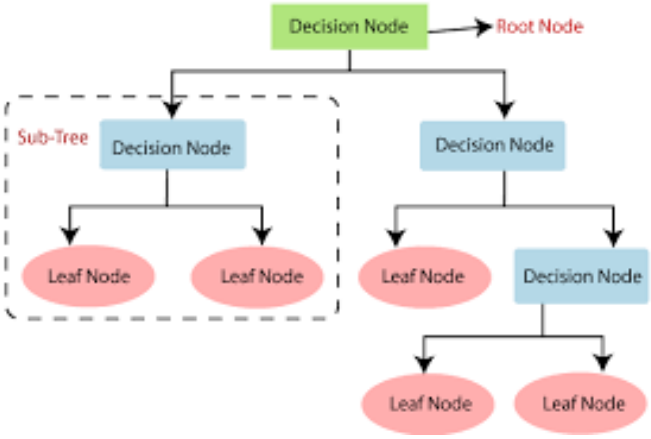

Fig 2 Decision Tree

2.2.2 Naive Bayes:

Naive Bayes Classifier is one of the simplest and most effective Classification algorithms which helps in building fast machine learning models that can make quick predictions. It is a probabilistic classifier, which means it predicts based on the probability of an object.

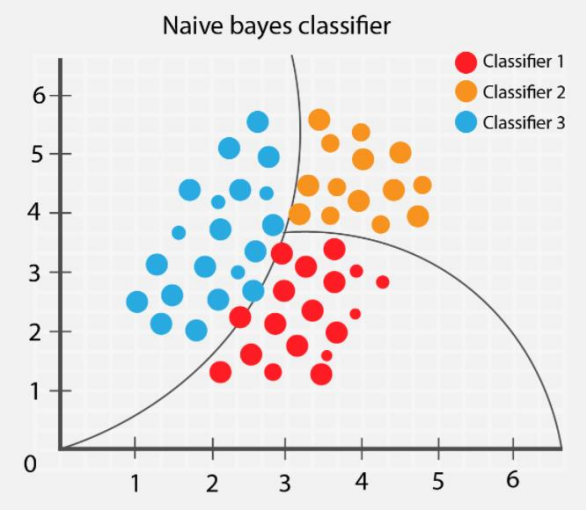

Fig 3 Naive Bayes

2.2.3 Random Forest:

"Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset." Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.

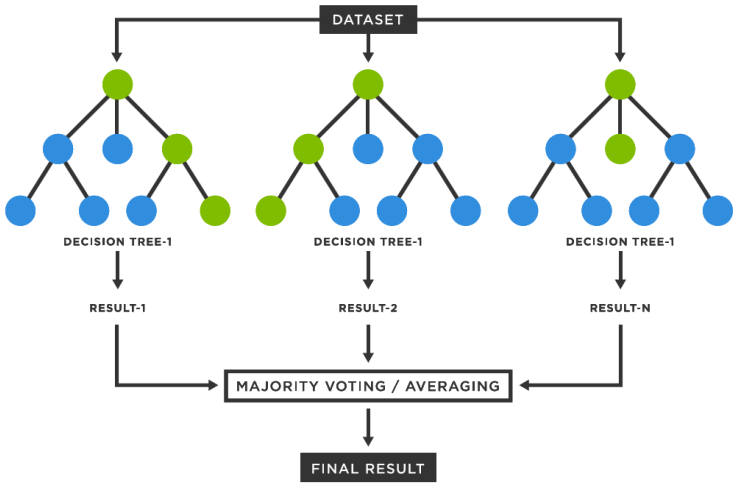

Fig 4 Random Forest

2.2.4 SVM:

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

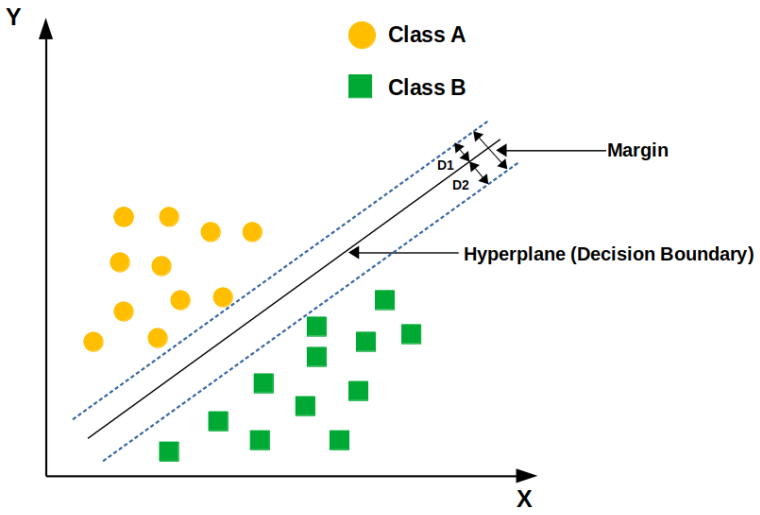

Fig 5 Support Vector Machine

2.3Gaps and Limitations in Existing Solutions

1. **Limited Accessibility**:
   Many existing spam detection systems are integrated within email providers, offering limited options for standalone, user-friendly tools.
2. **Complex Interfaces**:
   Current solutions often lack intuitive interfaces, making them challenging for non-technical users to operate.
3. **Focus on Advanced Models**:
   While deep learning models achieve high accuracy, they require significant computational resources and are impractical for lightweight applications.
4. **Dependence on Metadata**:
   Some systems rely heavily on email metadata (e.g., sender information, headers) rather than focusing on textual content, limiting their adaptability to text-based spam detection.
5. **Lack of Voice Feedback**:
   Few solutions enhance user experience through features like voice feedback, which can make tools more accessible and engaging.
6. **Scalability Challenges**:
   Many models are either designed for large-scale enterprise systems or small datasets, lacking versatility across different scales.

How This Project Addresses These Gaps

1. **Standalone Applications:**
   The project offers both a Tkinter-based desktop GUI and a Streamlit-based web app, providing accessible and versatile tools for users.
2. **Intuitive Design**:
   The interface is designed for ease of use, allowing users to classify emails with minimal technical expertise.
3. **Lightweight Machine Learning**:
   By using the Multinomial Naive Bayes algorithm, the project balances accuracy and computational efficiency, making it suitable for real-time applications.
4. **Content-Based Classification**:
   The model focuses on analyzing the textual content of emails, ensuring adaptability to various datasets and spam patterns.
5. **Voice Feedback**:
   Voice feedback in the desktop application enhances accessibility and user engagement, making it more interactive.
6. **Scalable Solution**:
   The system can handle moderate datasets efficiently and can be extended for larger-scale applications with minor adjustments.

CHAPTER 3

Proposed Methodology

3.1 System Design

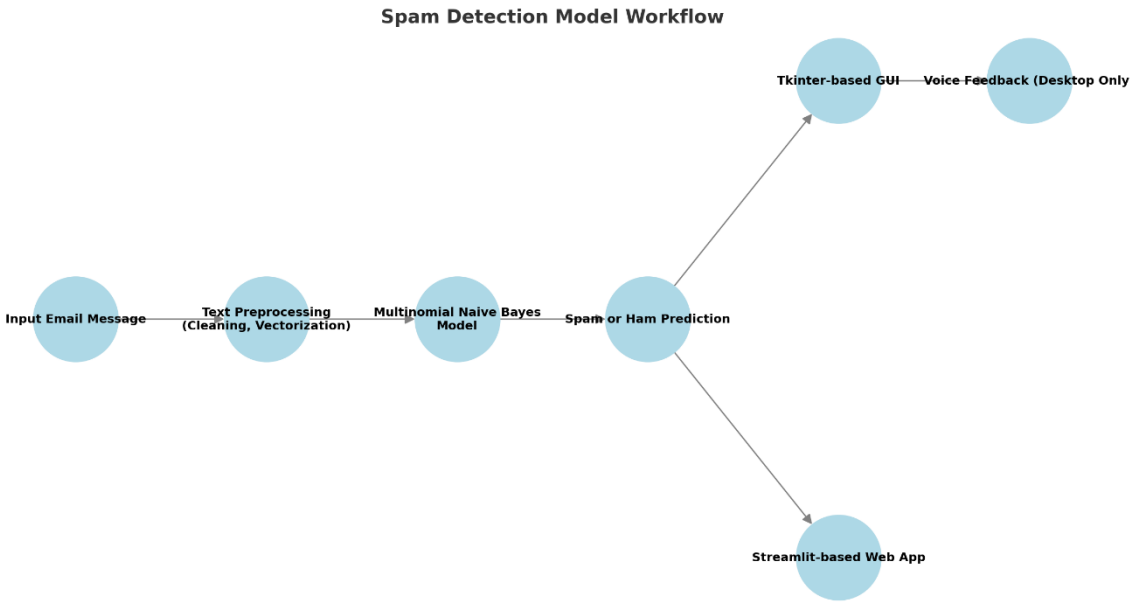

Fig 6 Spam Detection Model using NLP

The diagram outlines a workflow for a spam detection model. Here's a detailed explanation of each process state:

1. **Input Email Message**:
   - This is the starting point of the workflow where an email message is provided as input.
   - The input could be a plain text email or a structured email dataset.
   - This step is responsible for gathering the raw data for analysis and spam detection.

2. **Text Preprocessing (Cleaning, Vectorization)**:
   - This step involves preparing the email text for the machine learning model by:
     - **Cleaning**: Removing unwanted characters, HTML tags, punctuation, special symbols, and stopwords.
     - **Tokenization**: Breaking the email text into smaller units like words or tokens.

        - **Vectorization**: Converting the text data into numerical format using techniques like Bag of Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), or word embeddings.
    - This ensures the data is in a suitable format for the next stage.

3. **Multinomial Naive Bayes Model**:
    - This is the core machine learning model used for classification.
    - The **Multinomial Naive Bayes** algorithm is well-suited for text data and is based on Bayes' Theorem with an assumption of independence between features.
    - The model uses preprocessed numerical data to learn patterns associated with spam and non-spam (ham) messages.

4. **Spam or Ham Prediction**:
    - After training the model, it classifies the email as either spam or ham based on its learned patterns.
    - The output is a prediction label (e.g., "spam" or "ham") and could also include a probability score indicating confidence in the prediction.

5. **Tkinter-based GUI**:
    - This is a graphical user interface created using the **Tkinter** library in Python.
    - It allows users to interact with the model locally on their desktop.
    - Users can input email text into the GUI and view the classification result (spam or ham) in real-time.

6. **Voice Feedback (Desktop Only)**:
    - This feature enhances accessibility by providing an auditory output of the classification result.
    - For example, the system might say "This is spam" or "This is not spam."
    - This is particularly useful for visually impaired users or for environments where auditory feedback is preferred.

7. **Streamlit-based Web App**:
    - This is an alternative deployment platform for the model using the Streamlit library.
    - It allows users to interact with the model via a web browser.
    - Users can input their email text through a user-friendly web interface and receive classification results.

- Streamlit is ideal for sharing the application with remote users over the internet.

3.2 Requirement Specification

3.2.1 Hardware Requirements:

1. **Processor**: Dual-core or higher (Intel i3 or equivalent, or better).
2. **Memory (RAM)**: At least 4 GB; 8 GB or higher recommended for better performance.
3. **Storage**: Minimum 500 MB of free disk space for project files, datasets, and libraries.
4. **Graphics**: Integrated graphics sufficient; no high-end GPU is required for this project.
5. **Peripherals**: Keyboard, mouse, and speakers (for voice feedback in the Tkinter GUI).

3.2.2 Software Requirements:

1. **Operating System**:
   - Windows 10/11, macOS, or Linux (Ubuntu or similar distributions).
2. **Programming Language**:
   - Python 3.8 or higher.
3. **Python Libraries and Tools**:
   - **Data Manipulation**: pandas, numpy.
   - **Machine Learning**: scikit-learn.
   - **Natural Language Processing**: CountVectorizer from scikit-learn.
   - **Serialization**: pickle.
   - **Visualization (for development)**: matplotlib, networkx (optional for diagrams).
4. **Graphical User Interfaces**:

- **Desktop Application**: Tkinter (pre-installed with Python).
- **Web Application**: Streamlit.

5. **Voice Feedback**:
   - **win32com.client** library (available in the pywin32 package for Windows).
6. **Integrated Development Environment (IDE)**:
   - **Optional**: PyCharm, Visual Studio Code, or Jupyter Notebook for development.
7. **Web Browser**:
   - For testing the Streamlit application.
8. **Other Requirements**:
   - **Package Manager**: pip or conda for installing dependencies.
   - **Text Editor**: Any code editor like Notepad++ or Sublime Text for quick edits.

CHAPTER 4

Implementation and Result

4.1 Snap Shots of Result:

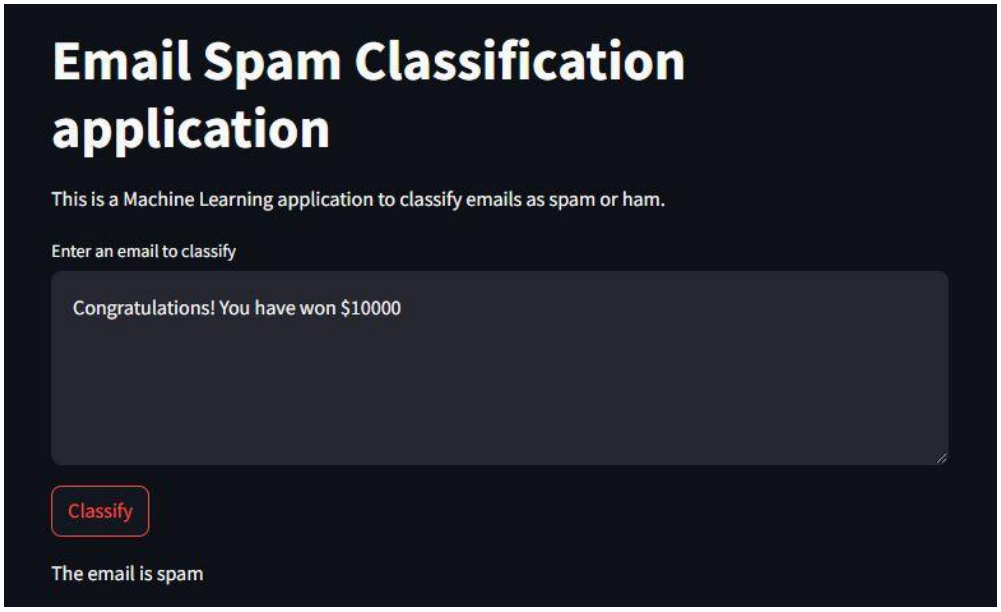

Fig 7 Spam email

Key Elements:

Title:

- "Email Spam Classification application" - This clearly states the purpose of the application.

Description:

- "This is a Machine Learning application to classify emails as spam or ham." - This explains the underlying technology and functionality.

Input Field:

- "Enter an email to classify" - This is where the user enters the email content they want to classify.

Example Input:

- "Congratulations! You have won $10000" - This is an example of an email that might be classified as spam.

Classification Button:

- "Classify" - When clicked, the application analyzes the entered email and provides a classification result.

Classification Result:

- "The email is spam" - This is the result displayed after the classification process.

Additional Features:

- "Manage app" - This leads to settings or management options for the application.

Fig 8 Not Spam email

Key Elements:

1. **Title:** "Email Spam Classification application" - Clearly states the purpose of the application.
2. **Description:** "This is a Machine Learning application to classify emails as spam or ham." - Explains the underlying technology and functionality.
3. **Input Field:** "Enter an email to classify" - A text area where users input the email content they want to analyze.
4. **Example Input:** "We are going for a day out... Are you in?" - An example of an email input.
5. **Classification Button:** "Classify" - When clicked, the application processes the input email and determines its spam status.
6. **Classification Result:** "The email is not spam" - The result displayed after the classification process.

4.2GitHub Link for Code:

https://github.com/heyy-vaishnavi/SMS-Spam-Detection-model

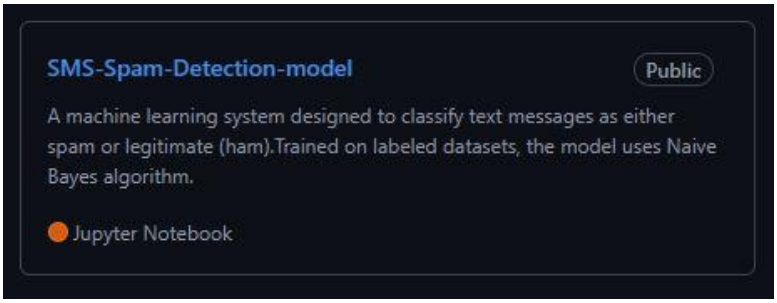

Fig 9 Github link

CHAPTER 5

Discussion and Conclusion

5.1 Future Work:

Some of the issues in spam detection are the presence of sarcastic text, multilingual data, and improper labelling of the datasets. Many researchers use APIs to gather data related to a given language and geographical area, there is a bias in the data collected through social media. Some studies employ raw data without much pre-processing, which results in duplicated features and lower classification performance. Some datasets exhibit a class imbalance, for example, the 'spam' class has many samples whereas the 'ham' class has a small number of samples.

There are a limited number of labelled datasets available for spam text, as well as a limited number of attributes available in these text datasets, which is a problem. For efficient research, a dataset with correct labelling is required, as is large computational power in the case of a large dataset. Only a few studies have used deep learning techniques and semantic approaches to detect spam. Exploring the use of multimodal content (text and images) from social media for social media would be a significant future challenge.

5.2 Conclusion:

I have described numerous strategies for spam text identification in depth in my systematic literature review on spam content detection and categorization. My research also investigated the various techniques for pre-processing, feature extraction, and spam text classification. This survey will assist researchers in conducting research in the field of social media spam detection as it highlights some of the best works done in this field. The various previous works on spam text pre-processing, feature extraction, and classification will aid researchers in determining the most appropriate strategies for their research in this area.

In future development, I'd like to include some other spam detection approaches, as well as their benefits and drawbacks.

REFERENCES

1. Kłosowski (2018).Kłosowski P. Deep learning for natural language processing and language modelling. Signal Processing: Algorithms, Architectures, Arrangements, and Applications (SPA); 2018. pp. 223–228. [Google Scholar]

2. Torfi et al. (2020).Torfi A, Shirvani RA, Keneshloo Y, Tavaf N, Fox EA. Natural language processing advancements by deep learning: a survey. 2020. https://arxiv.org/abs/2003.01200

3. Venkatraman, Surendiran & Arun Raj Kumar (2020).Venkatraman S, Surendiran B, Arun Raj Kumar P. Spam e-mail classification for the Internet of Things environment using semantic similarity approach. The Journal of Supercomputing. [Google Scholar]

4. Jancy Sickory Daisy & Rijuvana Begum (2021).Jancy Sickory Daisy S, Rijuvana Begum A. Smart material to build mail spam filtering technique using Naive Bayes and MRF methodologies. [Google Scholar]

5. Spam text classification techniques https://pmc.ncbi.nlm.nih.gov/articles/PMC8802784/#ref-93

6. Machine learning models: https://ijrpr.com/uploads/V3ISSUE11/IJRPR8167.pdf

7. https://ieeexplore.ieee.org/document/10170187